

Image Generation Using Diffusion Models (DDPM)

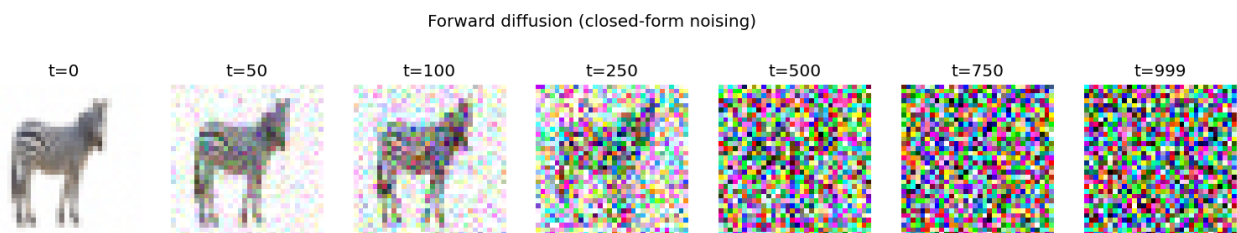
Fahad Khalid | MSDS25025 | Deep Learning - Assignment 5 (Bonus)

1. Overview

A Denoising Diffusion Probabilistic Model (DDPM) was implemented in PyTorch and trained to generate animal images. Five visually distinct classes were selected from the provided 15-class dataset (Zebra, Tiger, Elephant, Panda, Dolphin), giving 663 training images. All images were resized to 32x32 and normalized to $[-1, 1]$ so the data distribution is symmetric about zero, matching the standard-normal noise used in the diffusion process. The model predicts the noise added at each step (epsilon-prediction) and generates images by reversing the noising process from pure noise.

2. Forward (Noising) Process

The forward process adds Gaussian noise across $T = 1000$ timesteps using the closed-form $x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \text{noise}$, rather than adding noise iteratively. A linear beta schedule from $1e-4$ to 0.02 was used. The figure shows an image progressively corrupted: clean at $t=0$ and indistinguishable from noise by $t=999$.



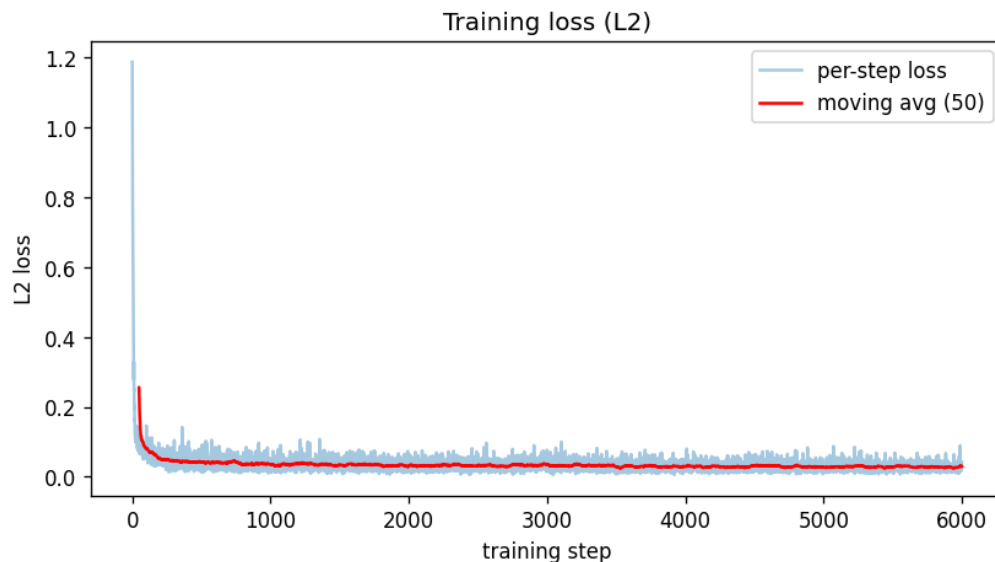
To confirm the process reaches a standard Gaussian, the noised-image statistics were measured: at $t=0$ mean 0.741 / std 0.421 (raw image), and at $t=999$ mean 0.006 / std 0.996 - effectively a standard normal distribution, as required.

3. Denoising Model

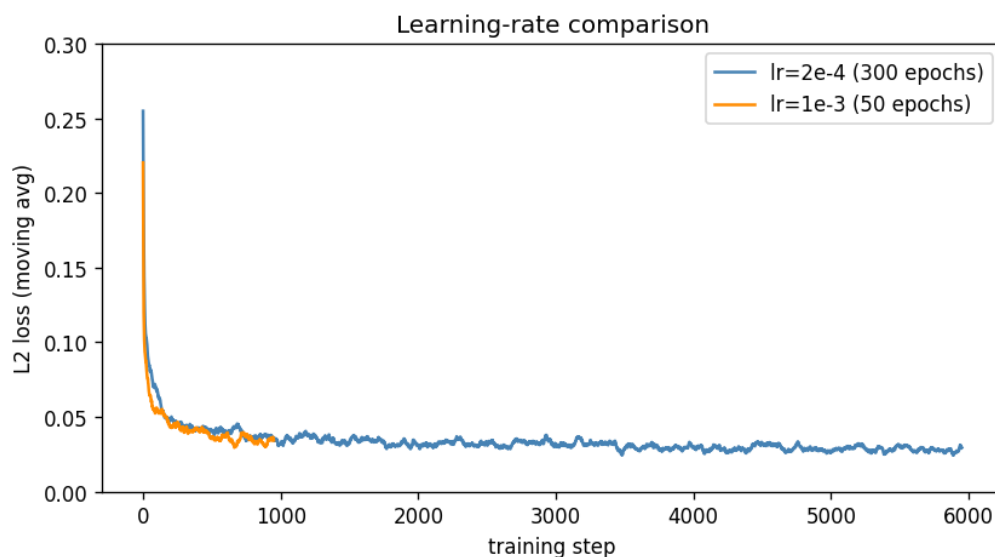
A small U-Net ($\sim 1.01M$ parameters) predicts the noise from a noised image and its timestep. Design choices: a sinusoidal timestep embedding injected into every block so one network handles all 1000 noise levels; GroupNorm instead of BatchNorm (batch statistics are unstable with small batches); SiLU activations and residual connections throughout; skip connections to preserve spatial detail lost in downsampling; and no output activation, since predicted noise is unbounded. The small capacity is deliberate to avoid memorization on limited data.

4. Training and Loss

The model was trained for 300 epochs with Adam (learning rate $2e-4$) and a custom L2 loss between predicted and true noise. The loss fell sharply from ~ 0.47 to ~ 0.03 within the first 20 epochs, then plateaued (final-step loss 0.0418, minimum 0.0055). The plateau does not reach zero, which is expected: with only 663 low-resolution images the model learns the general noise structure but cannot perfectly fit every sample - desirable, since the goal is generation, not memorization. The per-step loss is naturally noisy because each step trains on a randomly sampled timestep.



Effect of learning rate: to study this, a second model was trained at $1e-3$ and compared against the main $2e-4$ run. The higher rate converged faster in early epochs, reaching a comparable plateau (~ 0.034) in only 50 epochs versus 300, with slightly higher step-to-step variance. Both rates were stable and neither diverged, indicating the model is fairly robust to learning rate in this range - the higher rate trades marginally noisier updates for faster convergence.



5. Generated Results

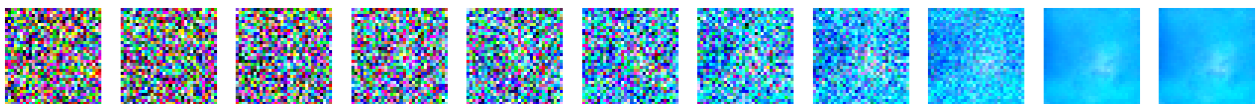
Images were generated by sampling pure Gaussian noise and running the reverse process for all 1000 timesteps. The samples are produced entirely from noise - they are new images, not reconstructions of training data.

Generated samples (from pure noise)



The samples are coherent rather than random noise: smooth color regions, characteristic backgrounds (e.g. the blue water/sky of the Dolphin class) and centered subject shapes. They are blurry and abstract rather than sharp animals - the expected outcome at this scale, where 663 images at 32x32 give enough signal for color and rough composition but not fine detail. The trajectory below shows a single image emerging from noise, mirroring the forward process.

Reverse process: noise (left) to image (right)



6. Problems and Solutions

Normalization: keeping images in $[0, 1]$ mismatches the zero-centered noise; normalizing to $[-1, 1]$ was essential for stable training. **Output activation:** tanh/sigmoid on the output harms training because noise is unbounded; a plain convolution was used. **Normalization layer:** BatchNorm was avoided in favor of GroupNorm due to small batches. **Sample quality:** the main limitation is dataset size and resolution; quality could improve with more images, higher resolution, or per-class (conditional) training, beyond this scope.

7. Conclusion

The implemented DDPM learns the forward and reverse diffusion processes and generates novel, coherent images from pure noise. The forward process reaches a standard Gaussian, the loss converges as expected, the learning-rate study shows robust behavior, and the reverse process produces structured samples consistent with the training distribution - a correct and working diffusion model within the constraints of a small dataset.